

Dona Lesma

Nome do arquivo: “lesma.x”, onde x deve ser c, cpp, pas, java, js, py2 ou py3

Dona Lesma é esportista e aventureira e definiu como objetivo deste verão alcançar o topo do muro do jardim em que vive. A cada dia, valente e metodicamente ela sobe exatamente uma certa distância (sempre a mesma a cada dia). Mas a cada noite enquanto dorme Dona Lesma escorrega para baixo uma outra distância (sempre a mesma a cada noite)...

Dadas a altura do muro, a distância que ela sobe a cada dia e a distância que ela desce a cada noite, ajude Dona Lesma a calcular quantos dias ela levará para chegar ao topo do muro.

Entrada

A primeira linha contém um inteiro A , a altura do muro. A segunda linha contém um inteiro S , distância que Dona Lesma sobe a cada dia. A terceira linha contém um inteiro D , a distância que Dona Lesma escorrega para baixo a cada noite.

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, o número de dias que Dona Lesma demorará para chegar ao topo do muro.

Restrições

- $1 \leq A \leq 10000$
- $1 \leq D < S \leq 10000$

Exemplos

Exemplo de entrada 1 4 2 1	Exemplo de saída 1 3
Exemplo de entrada 2 12 5 2	Exemplo de saída 2 4
Exemplo de entrada 3 10000 100 50	Exemplo de saída 3 199

Palavras Cruzadas

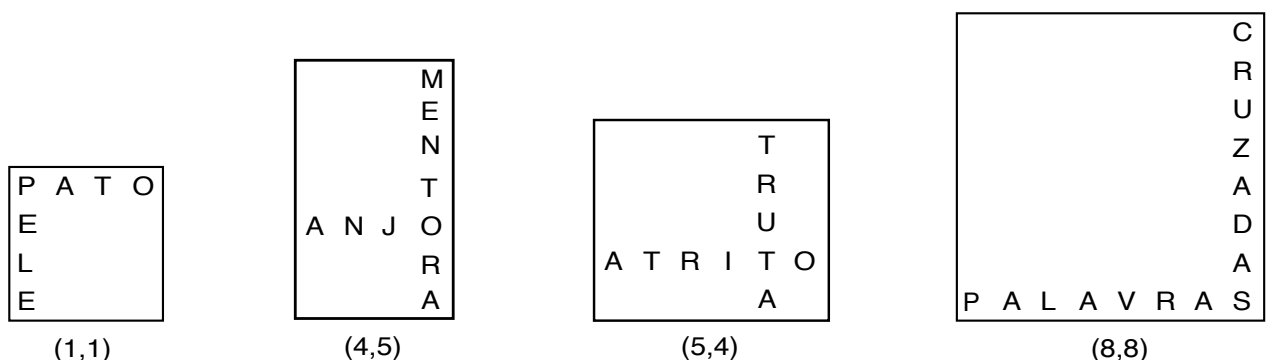
Nome do arquivo: “**cruzadas.x**”, onde **x** deve ser **c**, **cpp**, **pas**, **java**, **js**, **py2** ou **py3**

Você já deve ter tentado completar um jogo de palavras cruzadas. Nesse jogo, o jogador deve descobrir um conjunto de palavras através de dicas fornecidas juntamente com um retângulo dividido em quadrados de mesmo tamanho, sendo a maioria quadrados em branco e alguns quadrados pretos. Cada linha e cada coluna formada pelos quadrados em branco deve ser preenchida por uma palavra, com uma letra em cada quadrado em branco. As palavras das linhas são chamadas de palavras horizontais, as palavras das colunas são chamadas palavras verticais. As palavras horizontais *cruzam* com as palavras verticais em uma letra comum às duas palavras, vindo daí o nome do jogo.

Nesta tarefa, dadas uma palavra horizontal e uma palavra vertical, você deve encontrar a *melhor* letra de cruzamento entre elas, que é definida como a letra que produz

1. o cruzamento mais à direita possível na palavra horizontal;
2. se há mais de uma possibilidade de cruzamento com a regra (1), a melhor letra de cruzamento é definida como a que produz o cruzamento mais abaixo possível na palavra vertical.

A figura abaixo ilustra alguns exemplos. Entre parênteses estão os índices da melhor letra de cruzamento. O índice de uma letra é a posição que ela ocupa na palavra, iniciando com 1 (primeira letra).



Entrada

A primeira linha da entrada contém a palavra horizontal. A segunda linha da entrada contém a palavra vertical.

Saída

Seu programa deve produzir uma única linha, contendo apenas dois inteiros descrevendo a melhor letra de cruzamento. O primeiro número deve ser o índice da letra de cruzamento na palavra horizontal, o segundo número deve ser o índice da letra de cruzamento na palavra vertical. Se não há letra de cruzamento possível, os dois inteiros devem ser iguais a -1 .

Restrições

- As palavras são compostas por letras maiúsculas não acentuadas, com comprimento de pelo menos uma letra e de no máximo 1000 letras.

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 20 pontos há exatamente uma possibilidade de cruzamento possível.
- Para um conjunto de casos de testes valendo 80 pontos adicionais não há restrições adicionais.

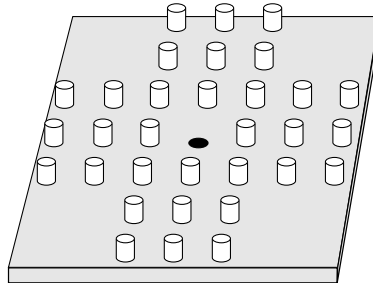
Exemplos

Exemplo de entrada 1 PATO PELE	Exemplo de saída 1 1 1
Exemplo de entrada 2 ANJO MENTORA	Exemplo de saída 2 4 5
Exemplo de entrada 3 MENTORA ANJO	Exemplo de saída 3 7 1
Exemplo de entrada 4 URUBU POLIVALENTE	Exemplo de saída 4 -1 -1

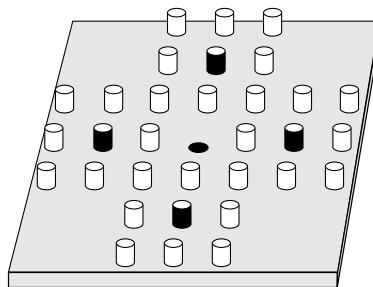
Jogo dos Pinos

Nome do arquivo: “`pinos.x`”, onde `x` deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

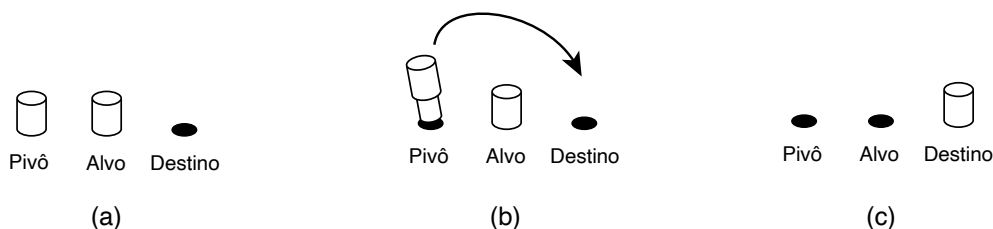
O Jogo dos Pinos é um quebra-cabeças que utiliza pinos e um tabuleiro com furos em forma de cruz. Inicialmente há apenas um furo vago, no centro do tabuleiro, e todos os outros furos contêm um pino como mostra figura abaixo.



O objetivo do jogo é remover os pinos do tabuleiro de forma que reste apenas um pino. Para remover um pino é necessário fazer um *movimento válido*, que é definido da seguinte maneira. O jogador deve escolher um pino, chamado *pivô*, e uma das quatro direções (acima, abaixo, esquerda, direita) de tal forma que o pivô tenha um outro pino, chamado *alvo*, como vizinho imediato na direção escolhida e que o pino alvo seja seguido, também na direção escolhida, por um furo vago (chamado de *destino*). A figura abaixo mostra os quatro possíveis pivôs da configuração inicial do jogo.



O jogador pode então fazer o pino pivô pular sobre o pino alvo e ocupar o furo destino, removendo o pino alvo do tabuleiro. A figura abaixo mostra um exemplo (a) antes, (b) durante e (c) depois de um movimento válido.



Dada uma configuração de pinos em um tabuleiro, escreva um programa para determinar o número de movimentos válidos possíveis na configuração dada.

Entrada

A entrada é composta por sete linhas, cada linha com exatamente sete caracteres. As linhas são identificadas por números de 1 a 7. Os dois primeiros caracteres e os dois últimos caracteres das linhas 1, 2, 6 e 7 são ‘-’ (hífen). Todos os outros caracteres são ou ‘o’ (letra o minúscula) representando um pino, ou ‘.’ (ponto) representando um furo.

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, o número de movimentos válidos na configuração da entrada.

Restrições

- A seção Entrada descreve as restrições.

Exemplos

Exemplo de entrada 1 --000-- --000-- 0000000 000.000 0000000 --000-- --000--	Exemplo de saída 1 4
Exemplo de entrada 2 --.0.-- --0.0-- ...0.. ...0.. 0.0.0.. --0.0-- --0.0--	Exemplo de saída 2 2

Dona Formiga

Nome do arquivo: “`formiga.x`”, onde `x` deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

Dona Formiga é uma ótima trabalhadora e todos os dias coleta muitas folhas para seu formigueiro. Mas no final de semana, quando todas as outras formigas estão descansando, ela gosta de se divertir escorregando pelos túneis do formigueiro.

O formigueiro de Dona Formiga tem muitos túneis e salões. Cada túnel conecta exatamente dois salões diferentes. Cada salão está a uma altura no formigueiro. Se existe um túnel ligando um salão I a um salão J e o salão I está a uma altura maior do que o salão J , então Dona Formiga pode escorregar do salão I para o salão J usando esse túnel.

Dados o mapa dos túneis do formigueiro, as alturas em que estão os salões e o salão de onde Dona Formiga quer partir, escreva um programa para determinar o maior número de salões que ela pode visitar (não contando o salão do qual ela parte), usando túneis exclusivamente para escorregar entre os salões.

Entrada

A primeira linha da entrada contém três inteiros S , T e P , respectivamente o número de salões, o número de túneis e o salão do formigueiro do qual Dona Formiga quer partir. Os salões são numerados de 1 a S . A segunda linha contém S números inteiros A_i , a altura em que o salão i está no formigueiro. Cada uma das T linhas seguintes contém dois inteiros I e J , indicando que há um túnel entre o salão I e o salão J .

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, o maior número de salões que Dona Formiga pode visitar (não contando o salão do qual ela parte), usando os túneis exclusivamente para escorregar entre os salões do formigueiro.

Restrições

- $1 \leq S \leq 200$
- $1 \leq T \leq S \times (S - 1)/2$
- $1 \leq P \leq S$
- $-1000 \leq A_i \leq 1000$ para $1 \leq i \leq S$
- $1 \leq I < J \leq S$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 20 pontos, $1 \leq S \leq 10$.
- Para um conjunto de casos de testes valendo 80 pontos adicionais, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
4 5 2 100 150 -50 200 1 2 2 4 1 4 3 4 1 3	2

Exemplo de entrada 2 4 5 3 100 150 -50 200 1 2 2 4 1 4 3 4 1 3	Exemplo de saída 2 0
Exemplo de entrada 3 4 5 4 100 150 -50 200 1 2 2 4 1 4 3 4 1 3	Exemplo de saída 3 3

Estrada

Nome do arquivo: “**estrada.x**”, onde **x** deve ser **c**, **cpp**, **pas**, **java**, **js**, **py2** ou **py3**

Para melhorar a integração com os países vizinhos, o Rei da Nlogônia decidiu que uma nova estrada será construída cruzando o país, da fronteira oeste à fronteira leste. O formato da estrada é uma única reta, que passará pelo centro de algumas cidades.

O Rei também decidiu que a construção será paga pelo Tesouro Real, mas cada cidade pela qual a estrada passar será responsável pela manutenção do trecho da estrada que constitui a *vizinhança da estrada* para aquela cidade. A *vizinhança da estrada* de uma cidade A é definida como todos os pontos da estrada que são mais próximos do centro da cidade A do que do centro de qualquer outra cidade.

Dados o comprimento total da estrada, de fronteira a fronteira, e as distâncias da fronteira oeste até os centros de cada cidade ao longo da nova estrada, escreva um programa para determinar qual a menor vizinhança de estrada entre as cidades pelas quais a estrada vai passar.

Entrada

A primeira linha da entrada contém um inteiro T , o comprimento total da estrada. A segunda linha contém um inteiro N , o número de cidades pelas quais a estrada vai passar. Cada uma das N linhas seguintes contém um inteiro X_i , indicando a distância da fronteira oeste até o centro da cidade i . Não há cidades nas fronteiras e cada centro de cidade tem uma localização distinta.

Saída

Seu programa deve produzir uma única linha, contendo um número real com duas casas após o ponto decimal, a menor vizinhança de estrada entre as cidades pelas quais a estrada vai passar.

Restrições

- $3 \leq T \leq 10^6$
- $2 \leq N \leq 10^4$
- $0 < X_i < T$, para $1 \leq i \leq N$
- $X_i \neq X_j$, para todo par $1 \leq i, j \leq N$.

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 10 pontos, $N = 2$.
- Para um conjunto de casos de testes valendo 90 pontos adicionais, nenhuma outra restrição.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
10 2 8 5	3.50

Exemplo de entrada 2	Exemplo de saída 2
10 3 7 6 8	1.00

Fotografia

Nome do arquivo: “**fotografia.x**”, onde **x** deve ser **c**, **cpp**, **pas**, **java**, **js**, **py2** ou **py3**

Chegou o final do ano, e os alunos resolveram dar um presente para a profa. Vilma: um quadro com a fotografia dos alunos classe. João está pesquisando os preços de molduras em um site da internet, e precisa de sua ajuda. Ele quer encontrar uma moldura tal que

- a fotografia seja inteiramente contida dentro na moldura; e
- a área da moldura que a fotografia não ocupa seja a menor possível; e
- uma moldura pode ser rotacionada para acomodar a fotografia, mas a fotografia deve ser acomodada de forma que seus lados sejam paralelos aos lados da moldura.

Dada as dimensões da fotografia e das molduras disponíveis, escreva um programa para ajudar João a escolher a melhor moldura, segundo os critérios acima.

Entrada

A primeira linha da entrada contém dois inteiros A e L indicando respectivamente a altura e largura da fotografia, em centímetros. A segunda linha da entrada contém um inteiro N , a quantidade de molduras disponíveis. As molduras são identificadas por números de 1 a N . Cada uma das N linhas seguintes contém dois inteiros X_i e Y_i indicando as dimensões em centímetros da moldura de número i , para $1 \leq i \leq N$.

Saída

Seu programa deve produzir uma única linha na saída, contendo um único número inteiro, o identificador da melhor moldura de acordo com os critérios acima. Se houver mais de uma moldura que satisfaz os critérios, o programa deve produzir o menor identificador entre as molduras que satisfazem os critérios. Se nenhuma moldura satisfaz os critérios, a linha deve conter -1 .

Restrições

- $1 \leq A \leq 100$
- $1 \leq L \leq 100$
- $1 \leq N \leq 100$
- $1 \leq X_i \leq 200$ para $1 \leq i \leq N$
- $1 \leq Y_i \leq 200$ para $1 \leq i \leq N$
- não há duas molduras com as mesmas dimensões

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
10 20 3 20 20 25 10 5 5	2

Exemplo de entrada 2 24 30 1 25 25	Exemplo de saída 2 -1
Exemplo de entrada 3 20 20 4 30 30 30 10 35 40 25 36	Exemplo de saída 3 1

Quebra-cabeças

Nome do arquivo: “quebra.x”, onde x deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

Um quebra-cabeças é composto por um tabuleiro composto por células quadradas organizadas em duas linhas de N colunas. Cada célula do tabuleiro pode conter uma ficha numerada, como na figura abaixo.

2		-3	5	
-1			2	3

As fichas podem ser deslizadas para uma célula não ocupada à direita ou à esquerda da posição corrente da ficha, mas a ordem das fichas, da esquerda para a direita, não pode ser alterada. Assim, na figura acima as fichas -3 e 5 podem ser movidas no máximo uma célula para a direita ou para a esquerda; já a ficha 3 pode ser movida somente para a esquerda, uma ou duas células. A figura abaixo ilustra algumas configurações possíveis para o quebra-cabeça da figura acima.

2		-3	5	
-1			2	3

(a)

2	-3			5
-1			2	3

(b)

		2	-3	5
-1	2	3		

(c)

O *valor* de uma configuração é a soma das multiplicações entre as fichas da primeira e da segunda linha do tabuleiro, para cada coluna (a ausência de ficha em uma célula é equivalente à presença de uma ficha de valor zero). Ou seja, para a configuração (a) acima, o valor é $2 \times -1 + -3 \times 0 + 5 \times 2 + 0 \times 3 = 8$; para a configuração (b), o valor é $2 \times -1 + -3 \times 0 + 0 \times 2 + 5 \times 3 = 13$; para a configuração (c), o valor é $0 \times -1 + 0 \times 2 + 2 \times 3 + -3 \times 0 + 5 \times 0 = 6$. O objetivo do quebra-cabeça é encontrar uma configuração com o maior valor possível.

Dada a descrição do tabuleiro e das fichas do quebra-cabeças, escreva um programa para determinar o maior valor possível que uma configuração pode ter.

Entrada

A primeira linha da entrada contém um inteiro N , o número de colunas do tabuleiro. Cada uma das duas linhas seguintes descreve as fichas de uma linha do tabuleiro. A linha da entrada inicia com um inteiro M indicando o número de fichas na linha do tabuleiro, seguido de M inteiros X_i indicando o valor e ordem das fichas.

Saída

Seu programa deve produzir uma única linha, contendo um único número inteiro, o valor máximo de uma configuração para o quebra-cabeças da entrada.

Restrições

- $1 \leq N \leq 500$
- $1 \leq M \leq N$
- $-100 \leq X_i \leq 100$ e $X_i \neq 0$ para $1 \leq i \leq M$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 20 pontos, $1 \leq N \leq 10$ e $M = N - 1$.
- Para um conjunto adicional de casos de testes valendo 30 pontos, $1 \leq N \leq 200$.
- Para um conjunto adicional de casos de testes valendo 50 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1 5 3 2 -3 5 3 -1 2 3	Exemplo de saída 1 19
Exemplo de entrada 2 6 4 -20 7 13 -2 5 1 -2 7 1 -1	Exemplo de saída 2 133

Caixeiro Viajante

Nome do arquivo: “caixeiro.x”, onde x deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

Paulo é um *caixeiro viajante*, que visita clientes nas cidades do reino da Nlogônia para demonstrar novos produtos da empresa para a qual trabalha.

Paulo está organizando uma viagem para visitar cada uma das N cidades do reino. A Nlogônia é imensa, as cidades são distantes, de forma que Paulo vai sempre usar transporte aéreo para ir de uma cidade a outra. Paulo conhece o tempo de voo entre cada par de cidades, mas não gosta de viajar de avião e quer minimizar o tempo de voo total para a viagem. No entanto, Paulo tem uma demanda peculiar quanto à ordem das cidades que vai visitar.

Na Nlogônia os “nomes” das cidades são números inteiros de 1 a N . Paulo deseja que a ordem das cidades que vai visitar obedecam à seguinte regra: quando Paulo visitar a cidade K , ou todas as cidades de nomes menores do que K já foram visitadas ou todas serão visitadas após K .

Por exemplo, se N é igual a três, Paulo pode visitar as cidades nas ordens 1, 2, 3 ou na ordem 3, 2, 1 ou na ordem 2, 1, 3, mas não pode visitá-las na ordem 1, 3, 2, pois quando visita a cidade 3, como 1 já foi visitada, 2 também deveria ter sido visitada.

Escreva um programa para determinar o tempo mínimo total de voo para a viagem de Paulo, iniciando em qualquer cidade, terminando em qualquer cidade, mas visitando cada cidade uma única vez.

Entrada

A primeira linha da entrada contém um inteiro N , o número de cidades. Cada uma das $N(N-1)/2$ linhas contém três inteiros A , B e T , indicando que o tempo de voo da cidade A para a cidade B é T (o tempo de voo da cidade B para a cidade A também é T).

Saída

Seu programa deve produzir uma única linha, contendo um único número inteiro, o tempo mínimo total de voo para a viagem de Paulo.

Restrições

- $2 \leq N \leq 1500$
- $1 \leq A < B \leq N$
- $1 \leq T \leq 1000$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 25 pontos, $1 \leq N \leq 10$.
- Para um conjunto adicional de casos de testes valendo 50 pontos, $1 \leq N \leq 20$.
- Para um conjunto adicional de casos de testes valendo 25 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
3 1 2 5 1 3 2 2 3 4	7

Exemplo de entrada 2	Exemplo de saída 2
4 1 2 15 1 3 7 1 4 8 2 3 16 2 4 9 3 4 12	31